

Module 16 – Generative AI Fundamentals

Introduction to Generative AI

What is Generative AI?

Generative AI (GenAI) is a type of Artificial Intelligence that creates new content such as text, code, images, audio, and videos based on user input (prompts). It learns patterns from existing data and generates human-like outputs.

Generative AI vs Traditional AI

Generative AI	Traditional (Discriminative) AI
Creates new content	Classifies or predicts data
Generates text, code, images	Identifies patterns and makes decisions
Examples: ChatGPT, GitHub Copilot	Examples: Spam filters, Image classifiers

Applications of Generative AI

- Text generation
- Code completion
- Image generation
- Chatbots and virtual assistants
- Content creation
- Software development
- Document summarization
- Language translation

History and Evolution of Generative AI

- **1960s:** Early chatbots (ELIZA)
- **2014:** Generative Adversarial Networks (GANs)
- **2020:** GPT-3 introduced advanced language generation
- **2022:** ChatGPT became widely popular
- **Present:** GitHub Copilot, GPT-4/5, multimodal AI, AI coding assistants

Context Engineering

What is Context Engineering?

Context Engineering is the process of providing relevant information (context) to an AI model so it can generate more accurate and useful responses.

Prompt Engineering vs Context Engineering

Prompt Engineering	Context Engineering
Focuses on writing effective prompts	Focuses on providing relevant information to the AI
Improves instructions	Improves understanding using additional context

Types of Context in AI

- **Instructions:** Rules or tasks for the AI.
- **User Input:** User's request or question.
- **Conversation History:** Previous interactions.
- **External Data:** Documents, databases, APIs, or files.

Context Window

A context window is the maximum amount of information (tokens) an AI model can process at one time.

Limitations

- Long conversations may exceed the context window.
- Older information may be forgotten.
- Only relevant context should be provided to improve accuracy and reduce errors.

Prompt Engineering

What is Prompt Engineering?

Prompt Engineering is the practice of designing effective prompts to obtain accurate and useful responses from AI models.

Prompting Techniques

Zero-shot Prompting

The AI performs a task without examples.

Example:

"Write a C# program to reverse a string."

Few-shot Prompting

The AI is given one or more examples before performing the task.

Example:

Provide sample input/output before asking the AI to generate similar code.

Chain-of-Thought Prompting

The AI is encouraged to explain its reasoning step by step before providing the final answer.

Best Practices

- Write clear instructions.
 - Provide sufficient context.
 - Specify the expected output format.
 - Break complex tasks into smaller steps.
 - Refine prompts based on results.
-

Ethical Considerations

- Avoid biased prompts.
 - Verify AI-generated information.
 - Protect sensitive and private data.
 - Use AI responsibly.
 - Review generated code before using it.
-

Coding Prompt Example

Prompt:

"Write a C# method to check whether a number is prime."

Introduction to GitHub Copilot

What is GitHub Copilot?

GitHub Copilot is an AI-powered coding assistant that helps developers write code faster by suggesting code, functions, documentation, and test cases.

Features of GitHub Copilot

- Code completion
- Function generation
- Boilerplate code generation
- Documentation generation
- Test case generation
- Code refactoring
- Intelligent code suggestions

How GitHub Copilot Works

GitHub Copilot acts as an AI pair programmer powered by OpenAI models. It analyzes comments, existing code, and project context to generate relevant code suggestions.

Supported IDEs

- Visual Studio Code
- Visual Studio
- JetBrains IDEs
- Neovim

Supported Languages

- C#
- Java
- Python
- JavaScript
- TypeScript
- C++
- Go
- Ruby
- PHP
- Many others

Setup and Configuration

Installing GitHub Copilot

1. Install Visual Studio Code.
2. Install the GitHub Copilot extension.
3. Sign in with a GitHub account.
4. Enable Copilot.
5. Start coding and accept suggestions using **Tab**.

Core Features and Capabilities

Code Suggestions

Provides real-time code completion while typing.

Writing Functions

Generates complete functions from comments or method names.

Generating Documentation

Automatically creates comments and documentation for methods and classes.

Refactoring Code

Suggests cleaner, optimized, and more readable code.

Creating Test Cases

Generates unit tests for existing code automatically.

Security and Ethical Considerations

Risks of AI-Generated Code

- Security vulnerabilities
 - Incorrect or hallucinated code
 - Outdated coding practices
-

Licensing and Attribution

AI-generated code may resemble publicly available code, so developers should verify licensing requirements before use.

Data Privacy

Some code or prompts may be processed by GitHub servers. Sensitive or confidential information should not be shared unnecessarily.

Responsible Use of Copilot

- Review generated code before deployment.
 - Validate correctness and security.
 - Avoid blindly accepting suggestions.
 - Follow coding standards and licensing policies.
-

Agentic AI Foundation

What is Agentic AI?

Agentic AI refers to AI systems that can independently plan, reason, make decisions, and perform tasks to achieve specific goals with minimal human intervention.

Agentic AI Lifecycle

1. Receive goal or task.
 2. Plan actions.
 3. Execute tasks.
 4. Evaluate results.
 5. Improve through feedback.
-

Foundations of Modern AI

Machine Learning (ML)

Machine Learning enables computers to learn from data without explicit programming.

Neural Networks

Neural networks are AI models inspired by the human brain that learn patterns from large datasets.

Real-World Applications

- Healthcare
- Finance
- Autonomous vehicles
- Recommendation systems
- Fraud detection
- Chatbots
- Software development
- Robotics

Summary

Generative AI creates new content such as text, code, and images using advanced AI models. Effective **Prompt Engineering** and **Context Engineering** improve AI output quality. **GitHub Copilot** acts as an AI coding assistant by generating code, documentation, refactoring suggestions, and test cases. Developers should use AI responsibly by verifying generated code, protecting sensitive information, and considering security and licensing concerns. Modern AI concepts such as **Machine Learning**, **Neural Networks**, and **Agentic AI** enable intelligent systems that support automation and real-world problem solving across many industries.